

lab01(SSU)

September 3, 2025

1 Part 1: Jupyter Notebooks

This webpage is called a Jupyter Notebook. A notebook is a place to write programs and view their results, and also to write text.

1.1 1.1. Text cells

In a notebook, each rectangle containing text or code is called a *cell*.

Text cells (like this one) can be edited by double-clicking on them. They're written in a simple format called [Markdown](#) to add formatting and section headings. You don't need to learn Markdown, but you might want to.

After you edit a text cell, click the “run cell” button at the top that looks like  or hold down **shift** + **return** to confirm any changes. (Try not to delete the instructions of the lab.)

Question 1.1.1. This paragraph is in its own text cell. Try editing it so that this sentence is the last sentence in the paragraph, and then click the “run cell” button or hold down **shift** + **return**.

1.2 1.2. Code cells

Other cells contain code in the Python 3 language. Running a code cell will execute all of the code it contains.

To run the code in a code cell, first click on that cell to activate it. It'll be highlighted with a little green or blue rectangle. Next, either press  or hold down **shift** + **return**.

Try running this cell:

```
[23]: print("Hello, World!")
```

Hello, World!

And this one:

```
[24]: print("\N{WAVING HAND SIGN}, \N{EARTH GLOBE ASIA-AUSTRALIA}!")
```

, !

The fundamental building block of Python code is an expression. Cells can contain multiple lines with multiple expressions. When you run a cell, the lines of code are executed in the order in which they appear. Every `print` expression prints a line. Run the next cell and notice the order of the output.

```
[25]: print("First this line,\nthen the whole ,\nand then this one.")
```

```
First this line,
then the whole ,
and then this one.
```

Question 1.2.1. Change the cell above so that it prints out:

```
First this line,
then the whole ,
and then this one.
```

1.3 1.3. Converting Cells

Change the following cell from a Markdown cell to a code cell and run the cell. You should get 6 as an output.

```
[26]: print(1 + 2 + 3)  # Change this to a code cell
```

```
6
```

1.4 1.4. Writing Jupyter notebooks

You can use Jupyter notebooks for your own projects or documents. When you make your own notebook, you'll need to create your own cells for text and code.

Question 1.4.1. Add a code cell below this one. Write code in it that prints out:

```
A whole new cell! 🎵🎵
```

(That musical note symbol is like the Earth symbol. Its long-form name is `\N{EIGHTH NOTE}`.)

Run your cell to verify that it works.

```
[27]: print("A whole new cell! 🎵🎵")
```

```
A whole new cell! 🎵🎵
```

You can also delete cells by clicking the trash can button in the menu bar.

Question 1.4.2. Delete the cell below this one.

Here's a summary of some useful keyboard shortcuts:

Action	Keyboard shortcut
Run cell + jump to next cell	shift + return
Save notebook	ctrl/cmd + s
Create new cell above/below	a/b
Delete cell	x
Switch to command mode	escape
Switch to edit mode	return

1.5 1.5. Errors

Python is a language, and like natural human languages, it has rules. It differs from natural language in two important ways: 1. The rules are *simple*. You can learn most of them in a few weeks and gain reasonable proficiency with the language in a semester. 2. The rules are *rigid*. If you're proficient in a natural language, you can understand a non-proficient speaker, glossing over small mistakes. A computer running Python code is not smart enough to do that.

Whenever you write code, you'll make mistakes. When you run a code cell that has errors, Python will sometimes produce error messages to tell you what you did wrong.

Errors are okay; even experienced programmers make many errors. When you make an error, you just have to find the source of the problem, fix it, and move on.

We have made an error in the next cell. Run it and see what happens.

```
[28]: print("This line is missing something.")
```

This line is missing something.

Note: In the toolbar, there is the option to click Run > Run All Cells, which will run all the code cells in this notebook in order. However, the notebook stops running code cells if it hits an error, like the one in the cell above.

You should see something like this (minus our annotations and the D8 error output):

The last line of the error output attempts to tell you what went wrong. The *syntax* of a language is its structure, and this **SyntaxError** tells you that you have created an illegal structure.

There's a lot of terminology in programming languages, but you don't need to know it all in order to program effectively. If you see a cryptic message like EOF, you can often get by without deciphering it. (Of course, if you're frustrated, ask a neighbor or a staff member for help.) However, our D8 error package helps explain any python errors and exceptions and their common causes, as well as resources to reference for additional help. In the example above, we see that one source of a **SyntaxError** has to do with parentheses.

Try to fix the code above so that you can run the cell and see the intended message instead of an error.

1.6 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a pdf file for you to submit. **Please save before exporting!**

```
[21]: !apt-get install texlive texlive-xetex texlive-latex-extra pandoc
      !pip install pypandoc

      # Connect Google Drive to Colab so you can access your files
      from google.colab import drive
      drive.mount('/content/drive')

      # should change the directory and file name matching to yours
      !jupyter nbconvert '/content/drive/MyDrive/Colab Notebooks/DS/lab01(SSU).ipynb'
      ↪--to pdf
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 35 not upgraded.
Requirement already satisfied: pypandoc in /usr/local/lib/python3.12/dist-
packages (1.15)
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
[NbConvertApp] WARNING | pattern '/content/drive/MyDrive/Colab
Notebooks/DS/lab01(SSU).ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.
```

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

<cmd> --help-all

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json
Show the application's configuration (json format)
Equivalent to: [--Application.show_config_json=True]

--generate-config
generate default config file
Equivalent to: [--JupyterApp.generate_config=True]

-y
Answer yes to any questions instead of prompting.
Equivalent to: [--JupyterApp.answer_yes=True]

--execute
Execute the notebook prior to export.
Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors
Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was specified, too.
Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin
read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'
Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout
Write notebook output to stdout instead of files.
Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace
Run nbconvert in place, overwriting the existing notebook (only relevant when converting to notebook format)
Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]

--clear-output
Clear output of current file and save in place, overwriting the existing notebook.
Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]

--coalesce-streams
Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]

--no-prompt
Exclude input and output prompts from converted document.
Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]

--no-input
Exclude input cells and output prompts from converted document.

This mode is ideal for generating code-free reports.

Equivalent to: [--TemplateExporter.exclude_output_prompt=True
 --TemplateExporter.exclude_input=True
 --TemplateExporter.exclude_input_prompt=True]
 --allow-chromium-download

Whether to allow downloading chromium if no suitable version is found on the system.

Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
 --disable-chromium-sandbox

Disable chromium security sandbox when converting to PDF..
 Equivalent to: [--WebPDFExporter.disable_sandbox=True]
 --show-input

Shows code input. This flag is only useful for dejavu users.
 Equivalent to: [--TemplateExporter.exclude_input=False]
 --embed-images

Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.
 Equivalent to: [--HTMLExporter.embed_images=True]
 --sanitize-html

Whether the HTML in Markdown cells and cell outputs should be sanitized..
 Equivalent to: [--HTMLExporter.sanitize_html=True]
 --log-level=<Enum>

Set the log level by value or name.
 Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
 Default: 30
 Equivalent to: [--Application.log_level]
 --config=<Unicode>

Full path of a config file.
 Default: ''
 Equivalent to: [--JupyterApp.config_file]
 --to=<Unicode>

The export format to be used, either one of the built-in formats
 ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
 or a dotted object name that represents the import path for an
 ``Exporter`` class
 Default: ''
 Equivalent to: [--NbConvertApp.export_format]
 --template=<Unicode>

Name of the template to use
 Default: ''
 Equivalent to: [--TemplateExporter.template_name]
 --template-file=<Unicode>

Name of the template file to use
 Default: None
 Equivalent to: [--TemplateExporter.template_file]
 --theme=<Unicode>

Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)
 Default: 'light'
 Equivalent to: [--HTMLExporter.theme]

--sanitize_html=<Bool>
 Whether the HTML in Markdown cells and cell outputs should be sanitized. This should be set to True by nbviewer or similar tools.
 Default: False
 Equivalent to: [--HTMLExporter.sanitize_html]

--writer=<DottedObjectName>
 Writer class used to write the results of the conversion
 Default: 'FilesWriter'
 Equivalent to: [--NbConvertApp.writer_class]

--post=<DottedOrNone>
 PostProcessor class used to write the results of the conversion
 Default: ''
 Equivalent to: [--NbConvertApp.postprocessor_class]

--output=<Unicode>
 Overwrite base name use for output files.
 Supports pattern replacements '{notebook_name}'.
 Default: '{notebook_name}'
 Equivalent to: [--NbConvertApp.output_base]

--output-dir=<Unicode>
 Directory to write output(s) to. Defaults to output to the directory of each notebook.
 To recover previous default behaviour (outputting to the current working directory) use . as the flag value.
 Default: ''
 Equivalent to: [--FilesWriter.build_directory]

--reveal-prefix=<Unicode>
 The URL prefix for reveal.js (version 3.x).
 This defaults to the reveal CDN, but can be any url pointing to a copy of reveal.js.
 For speaker notes to work, this must be a relative path to a local copy of reveal.js: e.g., "reveal.js".
 If a relative path is given, it must be a subdirectory of the current directory (from which the server is run).
 See the usage documentation (<https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slideshow>) for more details.
 Default: ''
 Equivalent to: [--SlidesExporter.reveal_url_prefix]

`--nbformat=<Enum>`

The nbformat version to write.

Use this to downgrade notebooks.

Choices: any of [1, 2, 3, 4]

Default: 4

Equivalent to: `[--NotebookExporter.nbformat_version]`

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes

'base', 'article' and 'report'. HTML includes 'basic', 'lab' and 'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
```

```
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```



```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use `--help-all`.